

Licence MIASHS première année

Rapport de projet informatique

SMART LYON

Projet réalisé pour le 23 mai 2026

Membres du groupe

BARA Hana 44001262

JAHOUACH Tayssir 45005476

LANOUAR Meriam 45016911

Liens importants

`https://github.com/hababara1-byte/projet2.git`

`https://document-follower--hababara1.replit.app`

Ci dessus notre site, vous pouvez y accéder en toute sécurité.

Table des matières

1	Introduction	4
2	Environnement de travail	4
2.1	Langage de programmation	4
2.2	Logiciels utilisés	4
3	Description du projet et objectifs	5
3.1	Description de notre projet	5
3.2	Objectifs	5
3.3	Descriptions des acteurs	6
3.4	Relation entre les acteurs	6
3.4.1	Relation entre l'habitant et le maire	6
3.4.2	Relation entre le maire et le dirigeant	6
3.5	Rôle de l'Intelligence Artificielle	7
4	Bibliothèques, Outils et Technologies	7
4.1	Les bibliothèques utilisées	7
4.1.1	Côté frontend (interface utilisateur)	8
4.1.2	Côté backend (serveur API)	8
4.2	Les outils de développement	8
4.3	Les technologies utilisées dans le projet	8
5	Travail réalisé	10
5.1	Fonctionnalités prévue	10
5.1.1	Scénario d'utilisation	10
5.1.2	Habitant	10
5.1.3	Maire	11
5.1.4	Dirigeant	12
5.2	Fonctionnalités non aboutis	13
5.3	Répartition du travail	14
6	Difficultés rencontrées	15
7	Bilan	15
7.1	Conclusion	15
7.2	Perspective	15
8	Webographie	16
9	Annexes	17
A	Cahier des charges	17
B	Exemple d'exécution du projet	17
C	Manuel utilisateur	17

1 Introduction

Avec l'essor du numérique et de l'intelligence artificielle, les villes cherchent à améliorer leur fonctionnement pour offrir une meilleure qualité de vie. Les enjeux majeurs : pollution, chômage, transports, infrastructures — exigent des outils capables d'analyser les données et d'éclairer la décision.

Dans ce cadre, nous développons une simulation de ville intelligente en JavaScript, intégrant une IA simple fondée sur des règles. Cette IA analyse des indicateurs urbains et propose des recommandations adaptées.

L'étude porte sur la ville de Lyon et met en scène trois acteurs : habitants, maire et dirigeant, chacun doté de fonctionnalités spécifiques. L'objectif est de simuler la gestion d'une ville intelligente

2 Environnement de travail

2.1 Langage de programmation

Le projet est développé intégralement en TypeScript, un langage de programmation open-source créé par Microsoft. TypeScript est une surcouche de JavaScript qui y ajoute un système de typage statique fort : chaque variable, paramètre de fonction et valeur de retour possède un type déclaré, ce qui permet de détecter les erreurs dès la phase de développement plutôt qu'à l'exécution.

TypeScript est utilisé sur les deux couches du projet :

Côté serveur (backend) le serveur API Express tourne sur l'environnement d'exécution Node.js 24, qui interprète le JavaScript compilé depuis TypeScript. Node.js permet d'exécuter du code JavaScript en dehors du navigateur, côté serveur.

Côté client (frontend) l'interface utilisateur est développée avec React, une bibliothèque JavaScript pour la construction d'interfaces graphiques. React utilise la syntaxe TSX (TypeScript + JSX), qui permet d'écrire des composants d'interface directement dans le code TypeScript sous une forme proche du HTML.

Le choix d'un seul langage pour toute la pile technique présente un avantage majeur : les types de données sont partagés et cohérents entre le frontend et le backend, ce qui réduit les erreurs d'intégration.

2.2 Logiciels utilisés

Replit (environnement de développement principal) L'ensemble du projet a été développé sur Replit, un environnement de développement intégré (IDE) accessible directement depuis un navigateur web, sans installation locale. Replit offre un terminal Linux, un éditeur de code, un gestionnaire de fichiers et une infrastructure d'hébergement cloud. Il permet également de déployer l'application en production en un clic, via une URL publique en HTTPS.

Visual Studio Code (éditeur de référence) Bien que le développement se soit fait sur Replit, l'interface de celui-ci s'inspire de Visual Studio Code, l'un des éditeurs de code les plus utilisés dans l'industrie, développé par Microsoft. Il intègre la coloration

syntactique, la complétion automatique et la détection d'erreurs TypeScript en temps réel.

Navigateur web (Google Chrome / Firefox) Le navigateur est utilisé comme environnement de test et de prévisualisation de l'interface. Les outils de développement intégrés (DevTools) permettent d'inspecter les requêtes HTTP, les erreurs JavaScript et le rendu des composants React.

PostgreSQL PostgreSQL est le système de gestion de base de données relationnelle (SGBDR) utilisé pour stocker toutes les données de la simulation : métriques urbaines, décisions, signalements citoyens, témoignages et avis. Il est hébergé directement dans l'infrastructure Replit.

Git (gestion de versions) Le versionnement du code est assuré par Git, intégré à Replit sous forme de points de sauvegarde (*checkpoints*) automatiques à chaque étape significative du développement.

3 Description du projet et objectifs

3.1 Description de notre projet

La ville choisie pour ce projet est Lyon, une grande métropole située dans le sud-est de la France. Elle est considérée comme la troisième plus grande ville française après Paris et Marseille. Lyon appartient à la région Auvergne-Rhône-Alpes et constitue un important centre économique, culturel et universitaire. La ville compte environ 520 000 habitants et possède une superficie d'environ 47,87 km².

La métropole lyonnaise regroupe quant à elle plus d'un million d'habitants, ce qui en fait une zone urbaine très dynamique mais également confrontée à plusieurs problématiques liées à la gestion d'une grande ville moderne. Lyon est reconnue pour son activité économique importante dans plusieurs secteurs tels que la santé, la chimie, les biotechnologies, le numérique et les transports. Cependant, comme de nombreuses grandes villes, elle fait face à différents défis urbains :

- La pollution de l'air,
- Les embouteillages,
- La saturation des transports en commun,
- Le coût élevé du logement,
- Ainsi que certaines difficultés économiques et sociales.

Ces problématiques rendent Lyon particulièrement intéressante pour notre projet de simulation de ville intelligente. En effet, les données de cette ville permettent à l'intelligence artificielle du programme d'analyser des situations réelles et de proposer des recommandations adaptées afin d'améliorer la qualité de vie des habitants et la gestion urbaine.

3.2 Objectifs

L'objectif principal du projet est de concevoir un système informatique capable de simuler la gestion d'une ville intelligente en l'occurrence Lyon en intégrant une intelligence artificielle simplifiée basée sur des règles. Ce programme utilise les structures de

données, les fonctions, les conditions et la gestion de fichiers pour représenter la ville sous forme de données numériques, analyser automatiquement ses problèmes urbains (pollution, chômage, transports) et sauvegarder l'état du système. Enfin, il orchestre les interactions hiérarchiques entre trois types d'utilisateurs distincts l'habitant, le maire et le dirigeant afin que l'IA puisse leur proposer des recommandations et des solutions adaptées à leur rôle pour améliorer la gestion de la ville.

3.3 Descriptions des acteurs

Le système développé dans ce projet repose sur trois acteurs principaux possédant chacun un rôle et des fonctionnalités spécifiques. Cette organisation permet de simuler le fonctionnement d'une ville intelligente de manière réaliste.

L'habitant représente un utilisateur simple du système. Son rôle principal est de consulter les informations concernant la ville et de visualiser les recommandations proposées par l'intelligence artificielle.

Le maire représente le gestionnaire local de la ville. Il intervient directement dans la gestion des problèmes urbains et dans l'amélioration des conditions de vie des habitants.

Le dirigeant représente le niveau stratégique du système. Il dispose d'une vue globale de la ville et prend les décisions importantes concernant les investissements et les politiques générales. L'intelligence artificielle aide le dirigeant en analysant les données urbaines et en proposant des recommandations concernant les transports, l'environnement, l'économie ou encore les infrastructures. Cependant, la décision finale reste toujours prise par le dirigeant.

3.4 Relation entre les acteurs

Le système repose sur une organisation hiérarchique permettant de reproduire le fonctionnement simplifié d'une gestion urbaine réelle. Chaque acteur possède un rôle précis ainsi qu'un niveau de responsabilité différent.

3.4.1 Relation entre l'habitant et le maire

Les habitants représentent les citoyens de la ville. Ils consultent les informations et les recommandations proposées par l'intelligence artificielle, mais ils doivent également respecter les décisions et les règles mises en place par le maire.

Les habitants peuvent donner leur avis sur les actions ou les dispositifs appliqués dans la ville, notamment concernant :

- Les transports ;
- L'environnement ;
- la pollution ;
- Les aménagements urbains.

Le maire prend en considération ces avis afin d'améliorer la gestion locale de la ville.

3.4.2 Relation entre le maire et le dirigeant

Le maire agit au niveau local mais reste sous l'autorité du dirigeant. Il doit respecter les orientations stratégiques, les investissements et les décisions générales décidées par celui-ci.

Le dirigeant possède une vision plus globale de la ville et peut valider ou refuser certaines propositions locales. Cette hiérarchie permet de conserver une organisation cohérente du système.

3.5 Rôle de l'Intelligence Artificielle

Dans notre système, l'intelligence artificielle sert d'appui à la décision pour l'ensemble des acteurs. Elle analyse l'état de la ville et transforme les données disponibles en recommandations exploitables. Les habitants reçoivent des conseils généraux pour améliorer leur quotidien, le maire s'appuie sur l'IA pour prioriser et gérer les problématiques locales, tandis que le dirigeant bénéficie d'orientations plus stratégiques, notamment en matière d'investissements. L'IA n'agit cependant jamais à la place des utilisateurs : les choix finaux reviennent toujours aux responsables humains, principalement le maire et le dirigeant.

L'IA que l'on a développée s'appuie sur un ensemble de règles conditionnelles (if/else). Cette logique permet de relier des indicateurs urbains précis à des pistes d'action concrètes. Elle privilégie la transparence : pour chaque recommandation, on peut retracer la règle qui l'a déclenchée, ce qui facilite la compréhension et la validation par les décideurs.

Le système prend en compte plusieurs indicateurs clés : population, superficie, taux de chômage, niveau de pollution, et peut être étendu à d'autres mesures au besoin. Après un contrôle de cohérence et un prétraitement minimal (seuils, normalisations simples), ces données alimentent les règles d'évaluation qui détectent des situations à risque ou des marges d'amélioration.

À partir des seuils définis, l'IA signale automatiquement des problématiques telles qu'un chômage élevé, une pollution importante, une densité excessive ou des infrastructures insuffisantes. Chaque alerte est accompagnée d'un niveau de priorité, ce qui aide à planifier les actions sans se disperser.

Pour chaque problème identifié, l'IA suggère un ensemble de mesures adaptées : développer les transports en commun pour réduire la congestion et la pollution, limiter le trafic routier dans les zones critiques, stimuler l'emploi local via des projets ciblés, renforcer les infrastructures lorsque la densité l'exige, ou encore investir dans des initiatives écologiques. Ces propositions sont modulables et peuvent être combinées, puis ajustées par les décideurs selon les contraintes budgétaires et politiques.

Les recommandations sont présentées différemment selon le profil :

- Pour les habitants : conseils pratiques et informations de sensibilisation ;
- Pour le maire : aide à la gestion opérationnelle et au pilotage des services ;
- Pour le dirigeant : analyses synthétiques et scénarios d'investissement à moyen terme.

L'IA joue un rôle d'assistant fiable, mais non autonome. Elle éclaire, hiérarchise et justifie, tandis que la décision reste humaine. Ce partage garantit à la fois la réactivité face aux données et la légitimité démocratique des choix retenus.

4 Bibliothèques, Outils et Technologies

4.1 Les bibliothèques utilisées

Bien que ce projet soit développé en TypeScript/JavaScript et non en C, il repose sur un ensemble de bibliothèques organisées en deux catégories.

4.1.1 Côté frontend (interface utilisateur)

Bibliothèque	Version	Rôle
React	18.x	Framework d'interface utilisateur basé sur des composants
React Query (TanStack)	5.x	Gestion des requêtes HTTP et du cache de données
Recharts	2.x	Génération de graphiques interactifs (historique des métriques)
Tailwind CSS	3.x	Framework CSS utilitaire pour la mise en page
shadcn/ui	–	Composants UI prêts à l'emploi (boutons, cartes, sliders...)
Lucide React	–	Bibliothèque d'icônes vectorielles
React Router	6.x	Gestion de la navigation entre les pages
Zod	3.x	Validation des schémas de données côté client

TABLE 1 – Bibliothèques utilisées côté frontend

4.1.2 Côté backend (serveur API)

Bibliothèque	Version	Rôle
Express	5.x	Framework serveur HTTP pour Node.js
Drizzle ORM	0.x	Interaction avec la base de données PostgreSQL
Drizzle-Zod	–	Génération automatique de schémas Zod depuis les tables Drizzle
Pino	–	Journalisation (logging) structurée des requêtes
node-postgres (pg)	–	Connexion au serveur PostgreSQL

TABLE 2 – Bibliothèques utilisées côté backend

4.2 Les outils de développement

4.3 Les technologies utilisées dans le projet

Le projet s'appuie sur une architecture **full-stack moderne** organisée en monorepo.

Langage principal : TypeScript TypeScript est utilisé sur l'ensemble du projet, aussi bien côté frontend que backend, ce qui garantit la cohérence des types de données entre les deux couches.

Architecture monorepo avec pnpm Workspaces Le code est structuré en plusieurs packages indépendants partageant des bibliothèques communes :

- `lib/db` : schéma et connexion à la base de données ;
- `lib/api-spec` : contrat d'API au format OpenAPI 3.1 ;

Outil	Rôle
pnpm	Gestionnaire de paquets performant, utilisé pour gérer le monorepo (workspace)
Vite	Bundler et serveur de développement ultrarapide pour le frontend React
TypeScript 5.9	Langage de programmation typé, surcouche de JavaScript
Orval	Génération automatique de hooks React Query et de schémas Zod à partir de la spécification OpenAPI
esbuild	Compilateur/bundler haute performance pour le serveur Node.js
ESLint	Analyse statique du code JavaScript/TypeScript pour repérer les erreurs et mauvaises pratiques
Drizzle Kit	Outil de migration et de synchronisation du schéma de base de données
Replit	Environnement de développement intégré en ligne (IDE cloud) utilisé pour le développement et le déploiement

TABLE 3 – Outils de développement utilisés

- `lib/api-client-react` : hooks React générés automatiquement ;
- `lib/api-zod` : schémas de validation générés automatiquement ;
- `artifacts/api-server` : serveur Express ;
- `artifacts/ville-intelligente` : interface React + Vite.

Base de données : PostgreSQL Les données de la simulation (métriques de la ville, décisions, signalements citoyens, témoignages, avis) sont stockées dans une base de données relationnelle PostgreSQL. Le schéma est défini en TypeScript avec Drizzle ORM.

Communication client-serveur : OpenAPI + codegen Le contrat d'API est défini une seule fois dans un fichier OpenAPI (`openapi.yaml`), puis Orval génère automatiquement les hooks React Query et les schémas Zod correspondants. Cela élimine les risques d'incohérence entre le frontend et le backend.

Intelligence artificielle : règles expertes Le système d'IA est basé sur un moteur de règles métier (sans apprentissage automatique). Il analyse les métriques urbaines (taux de chômage, pollution, densité) et les signalements citoyens pour produire des analyses de situation, des recommandations d'actions et des solutions aux problèmes signalés, ainsi que des plans d'investissement avec budget estimé.

Déploiement : Replit Cloud L'application est déployée sur l'infrastructure cloud de Replit, accessible via une URL publique en HTTPS.

5 Travail réalisé

5.1 Fonctionnalités prévue

5.1.1 Scénario d'utilisation



FIGURE 1 – Première interface du site

1. L'utilisateur lance le programme.
2. Il choisit un rôle :
 - habitant
 - maire
 - dirigeant
3. Le système charge les données de la ville.
4. L'utilisateur interagit avec le système.
5. L'IA analyse les données et propose des solutions.
6. Les données peuvent être modifiées et sauvegardées :
 - Modifier les données de la ville :
 - taux de chômage
 - pollution
 - population (optionnel)
 - Lancer l'analyse de l'IA
 - Appliquer des décisions locales

5.1.2 Habitant

L'habitant occupe la place d'un utilisateur simple et d'un citoyen au sein du système de la ville intelligente. Ses possibilités d'action sont volontairement limitées puisqu'il ne dispose d'aucun pouvoir de modification sur les données de la ville et ne peut pas prendre de décisions directes sur le système.

Ses fonctionnalités se concentrent sur la consultation et l'expression citoyenne : l'habitant a la possibilité d'accéder de manière transparente aux informations publiques qui concernent l'état général de la ville de Lyon. Parallèlement, il peut consulter les analyses globales ainsi que les recommandations et les conseils généraux générés par l'intelligence artificielle pour les citoyens.

Visualiser les recommandations de l'intelligence artificielle est une importance particulière pour l'habitant. Bien qu'il n'ait pas le pouvoir d'agir sur les commandes du système ou de modifier les indicateurs de la métropole lyonnaise, l'habitant n'est pas un témoin passif. L'interface lui permet de voir concrètement les conseils généraux et les analyses que l'IA génère en temps réel.

En visualisant ces recommandations, le citoyen peut comprendre la logique des suggestions de l'IA face aux problèmes de la vie quotidienne, comme les pics de pollution, la gestion des transports en commun ou l'aménagement des quartiers. Cette transparence lui donne toutes les clés nécessaires pour analyser la situation de sa ville. C'est précisément grâce à cette visualisation que l'habitant peut ensuite formuler un avis critique et éclairé sur les stratégies de l'IA. Il peut ainsi juger si les solutions proposées par le système sont adaptées à sa réalité terrain, avant de transmettre indirectement son ressenti au maire pour orienter la gestion locale.

L'expression de son avis : Bien qu'il doit respecter les règles et les décisions mises en place par la municipalité, l'habitant a le droit de donner son avis critique sur les stratégies adoptées par l'intelligence artificielle ainsi que sur les actions ou les dispositifs appliqués dans la commune. Ces retours citoyens peuvent concerner des domaines du quotidien comme la gestion de la pollution, l'environnement, le réseau de transports ou encore les différents aménagements urbains.

La limite d'action de l'Habitant : L'habitant est conçu pour être un utilisateur simple du système, ce qui signifie que son rôle est purement consultatif et d'observation. Sa principale limite réside dans le fait qu'il ne possède absolument pas le droit de modifier les données de la ville de Lyon, ni de prendre la moindre décision directe sur le fonctionnement du programme. Sur le plan technique et informatique, cette restriction sert à préserver l'intégrité et la cohérence de la simulation de ville intelligente. Si un habitant pouvait modifier à sa guise les variables comme le taux de chômage ou le niveau de pollution, le système perdrait sa fiabilité, et l'intelligence artificielle ne reposerait plus sur des bases concrètes pour travailler.

Sur le plan fonctionnel, cette limite permet de respecter la hiérarchie logique d'une gestion urbaine réelle. L'habitant subit et observe l'environnement de la ville sans pouvoir en manipuler les commandes directes, ce qui sépare nettement son statut de celui du maire (gestionnaire local) ou du dirigeant (décisionnaire stratégique). Cependant, cette limite ne le rend pas passif : ne pouvant pas agir lui-même sur la machine, il est obligé de passer par les outils de dialogue mis à sa disposition. C'est précisément parce qu'il fait face à cette impossibilité de modifier le système qu'il doit utiliser sa voix pour donner son avis, transmettant ainsi ses remarques au maire pour espérer influencer indirectement la gestion de la commune.

5.1.3 Maire

Le maire représente le gestionnaire local de la ville. Il intervient directement dans la gestion des problèmes urbains et dans l'amélioration des conditions de vie des habi-

tants. Contrairement à l'habitant, il possède un pouvoir d'action direct sur le système pour traiter les problèmes urbains de proximité et améliorer les conditions de vie des administrés. Ses fonctionnalités se situent à l'échelle locale, ce qui signifie qu'il ne gère pas les investissements majeurs ni les orientations nationales.

La modification et la mise à jour des données locales : Le maire est habilité à intervenir directement sur les indicateurs de la ville en modifiant certaines données clés. Il peut par exemple actualiser le niveau de pollution de l'air ou ajuster le taux de chômage en fonction de l'évolution de la situation.

Le pilotage et la consultation de l'IA : C'est le maire qui a la responsabilité de lancer manuellement l'analyse des données urbaines par l'intelligence artificielle. Une fois cette analyse effectuée, il peut consulter les recommandations spécifiques proposées par le système pour résoudre les problématiques locales.

La prise de décision locale et la prise en compte des citoyens : Le maire utilise les suggestions de l'IA pour appliquer des décisions concrètes à l'échelle de la municipalité. Pour optimiser sa gestion, il prend également en considération les avis et les retours exprimés par les habitants afin d'ajuster ses actions de terrain.

La limite de responsabilité du Maire : Bien que le maire soit le gestionnaire direct de la commune et possède le pouvoir d'agir concrètement sur les problèmes urbains du quotidien, ses capacités d'action s'arrêtent dès que l'on touche aux grandes orientations de la ville. Sa principale limite réside dans le fait qu'il ne possède pas une vision stratégique globale du système. Sur le plan fonctionnel et décisionnel, cela signifie qu'il lui est totalement impossible de prendre des décisions d'envergure nationale ou d'engager des investissements majeurs par lui-même.

Cette restriction s'explique par la structure hiérarchique même du programme : le maire agit exclusivement au niveau local, mais il reste sous l'autorité directe du dirigeant. Il ne peut pas concevoir l'avenir de la métropole de manière autonome et se retrouve dans l'obligation de respecter strictement les orientations stratégiques, les budgets et les choix généraux validés par son supérieur. Le maire propose des solutions locales et applique des corrections de terrain (comme ajuster la pollution ou le chômage), mais il dépend entièrement du dirigeant pour l'approbation de ses projets et le financement des grandes infrastructures. C'est une limite qui sépare nettement la gestion opérationnelle de proximité (le maire) de la gouvernance financière et politique globale (le dirigeant).

5.1.4 Dirigeant

Le dirigeant représente le niveau stratégique du système. Il dispose d'une vue globale de la ville et prend les décisions importantes concernant les investissements et les politiques générales. Il bénéficie d'une autorité hiérarchique sur le maire, qui doit obligatoirement respecter ses directives générales. Disposant d'une vue d'ensemble et globale de la métropole, il est le décisionnaire ultime du projet :

La supervision et l'accès aux données de la ville : Le dirigeant se situe au sommet de l'organisation hiérarchique du système. Sa première grande fonctionnalité est de consulter l'état global et macroscopique de la ville de Lyon. Contrairement au maire qui reste

focalisé sur les indicateurs de proximité, le dirigeant possède une vue d'ensemble complète qui lui permet de suivre l'évolution générale de la métropole. Pour parfaire cette vision, il a le privilège d'accéder directement à toutes les analyses approfondies qui ont été réalisées par l'intelligence artificielle. Cela lui permet de comprendre les dynamiques de la ville en s'appuyant sur des données textuelles et chiffrées minutieusement triées par le programme.

L'aide à la décision et l'orientation des investissements : En tant que décideur de haut niveau, le dirigeant reçoit des recommandations stratégiques directement formulées par l'intelligence artificielle. Ces conseils ne sont pas de simples avertissements locaux, mais de véritables orientations politiques portant sur des secteurs majeurs comme l'économie, l'environnement, les transports ou encore les infrastructures urbaines. C'est en se basant sur ces recommandations que le dirigeant peut accomplir sa fonctionnalité la plus importante : choisir les secteurs prioritaires d'investissement. Il décide ainsi où l'argent et les ressources de la ville doivent être injectés en priorité pour résoudre les grands défis lyonnais comme la pollution ou la saturation des transports.

L'arbitrage et le pouvoir de décision finale : Bien que l'intelligence artificielle soit un outil extrêmement puissant pour analyser la ville, elle ne remplace jamais l'humain. Le dirigeant possède la fonctionnalité exclusive de valider ou de refuser les propositions de l'IA. Il agit comme un filtre critique : si une suggestion technique de la machine ne correspond pas à sa vision politique ou aux réalités budgétaires, il a le pouvoir de la rejeter. De la même manière, cette autorité lui permet d'arbitrer les demandes du maire. Au bout du compte, c'est le dirigeant qui prend les décisions finales concernant l'amélioration globale de la ville. Il est le seul maître à bord pour valider les transformations majeures de la métropole.

La limite de dépendance et de responsabilité du Dirigeant : La principale limite du dirigeant réside dans sa dépendance vis-à-vis des autres éléments du système pour pouvoir gouverner. Bien qu'il possède le pouvoir de décision finale, il ne peut pas inventer de solutions magiques tout seul : sa vision et ses choix dépendent entièrement des données urbaines récoltées sur le terrain et des analyses fournies par l'intelligence artificielle. Si l'IA ne lui soumet pas de suggestions ou si les données de la ville sont fausses, le dirigeant se retrouve bloqué ou risque de prendre de mauvaises décisions stratégiques.

De plus, une autre limite importante est liée à l'application concrète de ses choix. Le dirigeant a une vision globale et macroscopique, ce qui signifie qu'il est déconnecté des réalités quotidiennes de la ville. Il a besoin du maire pour appliquer ses orientations à l'échelle locale et a besoin que les habitants respectent ses règles. Il ne peut pas gérer les problèmes de proximité lui-même. Sa position de décideur ultime le rend donc totalement dépendant de la structure descendante du projet : sans l'IA pour l'aiguiller, sans le maire pour exécuter ses ordres sur le terrain, et sans les citoyens, son pouvoir stratégique reste purement théorique.

5.2 Fonctionnalités non aboutis

L'interaction sociale et l'espace communautaire citoyen : Dans la version actuelle du programme, chaque habitant utilise la plateforme de manière totalement isolée et fermée. Une fonctionnalité particulièrement intéressante qui n'a pas pu être développée

aurait été la création d'un espace communautaire ou d'un forum virtuel propre à la métropole lyonnaise. Dans une véritable ville intelligente, les citoyens ne se contentent pas de regarder des données : ils communiquent entre eux, débattent des décisions du maire et s'organisent pour faire remonter des requêtes collectives. Sur le plan informatique, faire interagir les habitants entre eux en simultané aurait nécessité d'intégrer de la programmation réseau avancée (comme les sockets en C) et de gérer des serveurs pour distribuer les messages, ce qui représentait une complexité technique bien trop élevée pour ce projet

Le système de signalement citoyen des anomalies de terrain : Pour l'instant, l'habitant a un rôle principalement axé sur la consultation des données de la ville et des recommandations de l'intelligence artificielle. Une fonctionnalité concrète de terrain qui n'a pas été développée est le signalement direct d'anomalies par les citoyens. On aurait pu imaginer une option permettant à un habitant d'envoyer une notification géolocalisée ou textuelle pour prévenir la mairie d'un problème quotidien précis, tel qu'une panne d'éclairage public, une dégradation sur la chaussée ou un dépôt sauvage d'ordures. Pour coder cela en langage C, il aurait fallu concevoir des structures de données dynamiques complexes, comme des files d'attente (files FIFO), permettant de stocker les signalements des habitants pour que le maire puisse les traiter et les résoudre un par un dans son menu.

Le système de notifications et d'alertes d'urgence instantanées : Actuellement, pour prendre connaissance de l'état de la ville ou voir les conseils de l'intelligence artificielle, l'habitant doit obligatoirement faire la démarche de lancer le programme et d'ouvrir son menu dédié. Une fonctionnalité manquante et très utile aurait été la mise en place d'un système d'alertes d'urgence automatisé. Si un événement critique survenait dans la simulation, comme un pic de pollution extrême à Lyon ou une saturation complète des lignes de transports en commun, le programme générerait instantanément un message d'alerte prioritaire à l'écran de l'habitant sans qu'il ait besoin de le demander. Cette interaction en temps réel a été écartée car elle exige de maîtriser le multi-threading en C pour faire tourner l'analyse de l'IA en arrière-plan pendant que l'utilisateur navigue dans les menus.

5.3 Répartition du travail

Pour la répartition du travail, on a d'abord réfléchi ensemble aux fonctionnalités, au design et à l'architecture générale, en partant de l'idée initiale proposée par Tayssir, que l'on a progressivement précisée au fil des échanges. La mise en œuvre technique s'est faite de manière collaborative, avec un développement porté en grande partie par Tayssir et Hana, tandis que Meriam assurait relectures, tests et ajustements. Le rapport a été rédigé collectivement. Meriam était particulièrement investie sur la syntaxe, la cohérence des formulations et l'uniformisation du style, ce qui a donné au document sa clarté finale. De son côté, Hana a pris en charge l'organisation sur Overleaf, la structuration des fichiers. Cette organisation, nous a permis d'avancer tout au long du projet.

6 Difficultés rencontrées

Au cours de la réalisation de ce projet, on a rencontré plusieurs difficultés, à la fois techniques et organisationnelles. Dès le départ, il a été compliqué de définir une idée claire et réalisable. Le choix d’une simulation de ville intelligente, portée par une intelligence artificielle simple, a exigé une longue phase de réflexion pour concilier originalité, respect des contraintes et faisabilité en langage C, un outil que l’on découvrait encore. Il a également fallu sélectionner un type d’IA cohérent avec nos objectifs.

La conception des rôles des utilisateurs (habitant, maire, dirigeant) et l’élaboration des recommandations de l’IA ont demandé de nombreux allers-retours avant d’aboutir à une structure satisfaisante. La gestion du temps a été l’un des enjeux majeurs, le projet comportant plusieurs étapes clés : compréhension du cahier des charges, conception du programme, développement en C, tests et corrections, puis rédaction du rapport. L’articulation entre les cours, le travail personnel et l’avancement du projet a parfois compliqué l’organisation, d’autant que certaines fonctionnalités — notamment la gestion des fichiers et l’implémentation d’une IA à base de règles — ont pris plus de temps que prévu.

Le travail en équipe a également représenté un défi. On a dû répartir les tâches, communiquer régulièrement, coordonner les différentes parties du projet et veiller à ce que tout le monde avance au même rythme. Des divergences d’idées sur l’organisation ou le développement ont parfois émergé ; elles ont nécessité des échanges constructifs pour dégager des décisions communes.

Enfin, l’accès au matériel informatique a constitué une difficulté supplémentaire. On n’avait pas toujours d’ordinateurs personnels à disposition pour travailler de manière régulière, ce qui a compliqué les phases de programmation, de test, de compilation et de correction. Ce manque d’accès permanent aux outils de développement a ralenti certaines étapes et nous a poussés à renforcer notre organisation collective pour maintenir l’avancement du projet.

7 Bilan

7.1 Conclusion

Ce projet montre, à petite échelle, comment une ville “intelligente” peut fonctionner en combinant des données, des règles simples en C et une IA qui propose des pistes d’action. On récupère quelques indicateurs clés, on les traite proprement et l’IA transforme tout ça en recommandations claires pour les différents acteurs. Elle reste volontairement simple et n’impose rien : elle aide à décider, elle n’agit pas à la place des humains.

7.2 Perspective

Cette première version prouve qu’avec une architecture sobre, on peut déjà obtenir des conseils utiles et faciles à comprendre. La suite logique serait d’élargir les données prises en compte, d’affiner les critères de priorité, d’améliorer les performances si le volume augmente, et pourquoi pas de tester un peu d’apprentissage automatique tout en gardant la transparence qui fait la force du prototype.

8 Webographie

Références

[GitHub] <https://github.com/hababara1-byte/projet2.git>

[Notre site] <https://document-follower--hababara1.replit.app>

[IA utilisé] <https://replit.com>

[IA utilisé] <https://claude.ai/login>

9 Annexes

Annexe A : Cahier des charges

Annexe B : Exemple d'exécution du projet

Annexe C : Manuel utilisateur